The Byzantine Generals Problem & Practical Byzantine Fault Tolerance

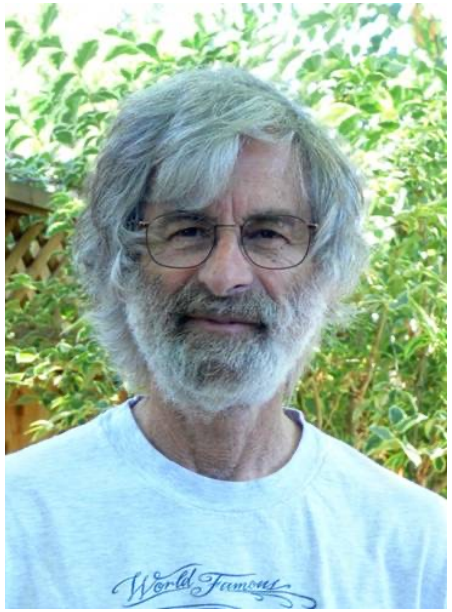
Presented by Daniel Lee

Byzantine Generals Introduction

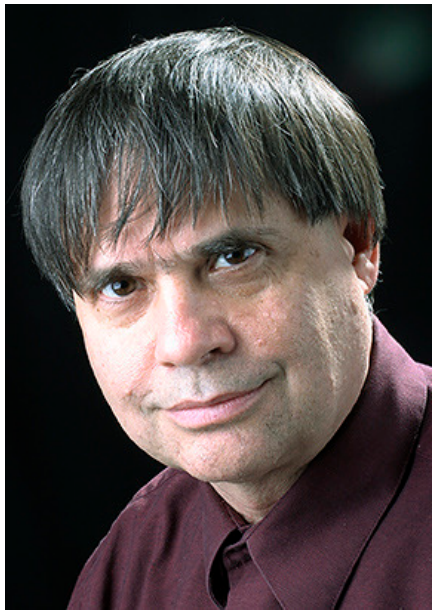
Written By:

- Leslie Lamport (1)
  - Brandeis University PhD in Math
  - Distributed Systems + Latex
  - 2013 Turing Award
- Robert Shostak (2)
  - Harvard PhD in CS
  - SRI International (CSL)
  - Byzantine Fault Tolerance
- Marshall Pease

1



2



Timeline

1978
Lamport claims to have published the first paper discussing arbitrary (Byzantine) faults

The Implementation of Reliable Distributed Multiprocess Systems

1978
Shostak et. Al formulate the problem and create a non cryptographic solution

SIFT: Design and analysis of a fault-tolerant computer for aircraft control

1980
Lamport, Pease and Shostak describe the non cryptographic solution and cryptographic solution to establish consensus in presence of Byzantine faults

Reaching Agreement in the Presence of Faults

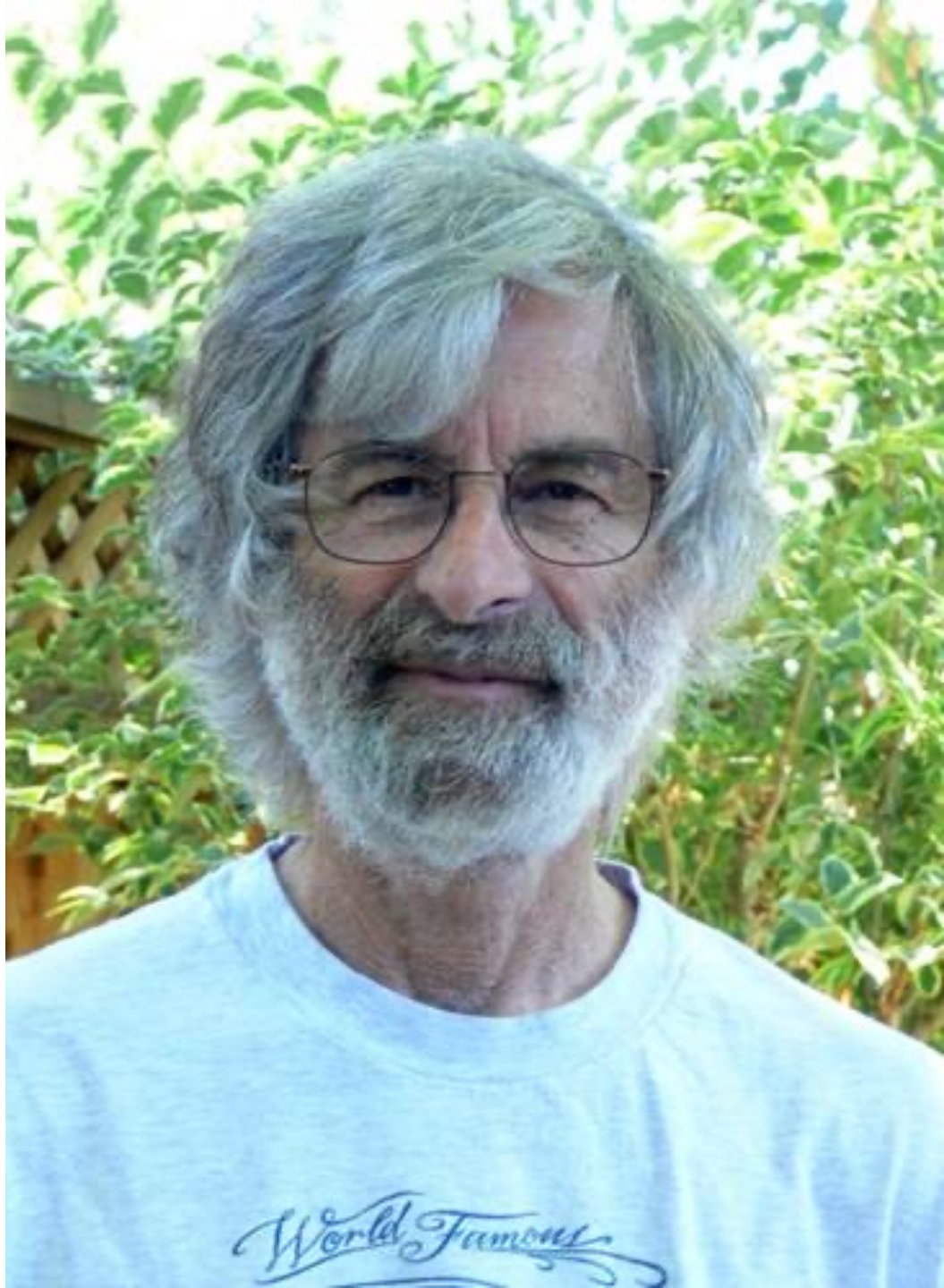
1982
Byzantine Generals paper is written

The Byzantine Generals Problem

*slide inspiration taken from https://www.cs.cornell.edu/courses/cs6410/2019fa/slides/18-distributed-systems-bgp.pdf

Byzantine Generals Background

“The popularity of the dining philosophers problem taught me that the best way to attract attention to a problem is to present it in terms of a story.”

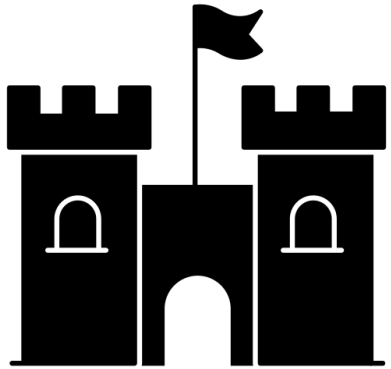


*quote taken from https://lamport.azurewebsites.net/pubs/pubs.html#byz

Byzantine Generals Background

“We imagine that several divisions of the Byzantine army are camped outside an enemy city, each division commanded by its own general. The generals can communicate with one another only by messenger. After observing the enemy, they must decide upon a common plan of action."



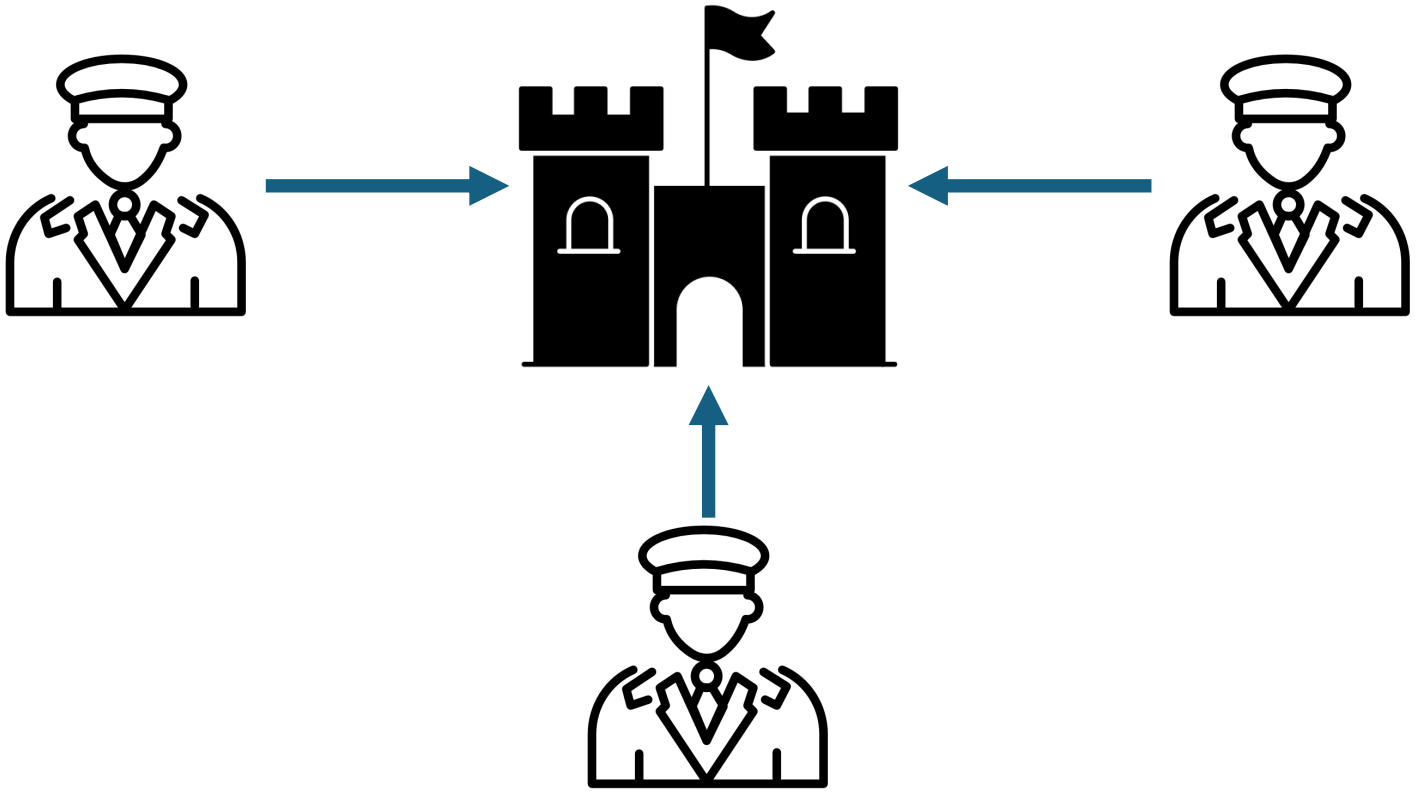






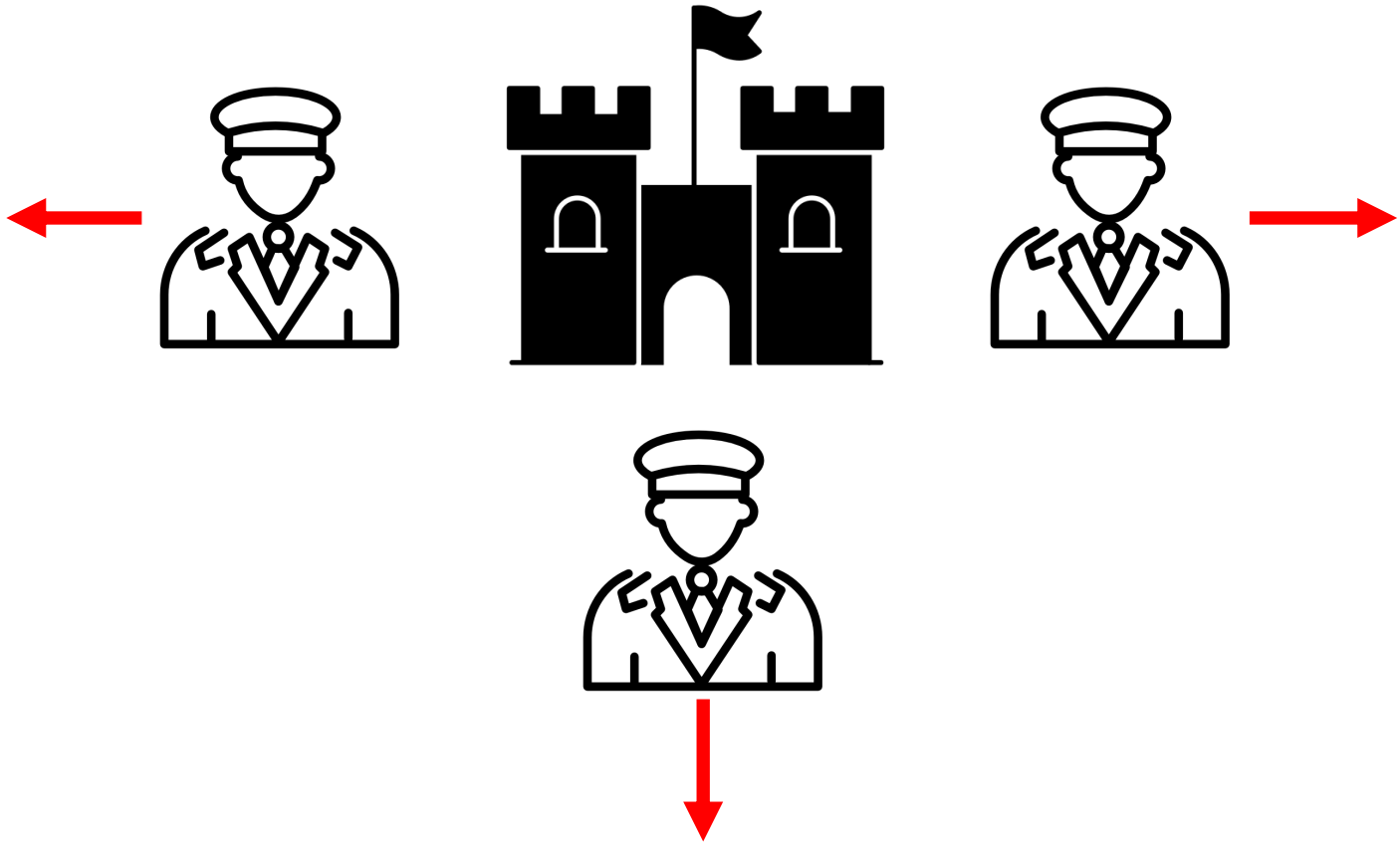
Byzantine Generals Background

- Generals need to reach consensus on plan
- Either all choose attack



Byzantine Generals Background

- Generals need to reach consensus on plan
- Either all choose attack
- Or all choose retreat



Generals Problem

- A: All loyal generals decide upon the same plan of action.
- B: A small number of traitors cannot cause the loyal generals to adopt a bad plan
- How do generals decide on a plan of action given [v_1, v_2, … v_n]
  - Where v_i = information from general i?

Generals Problem

- Every loyal general must obtain the same information [v_1, v_2, … v_n]
  - Rewritten: for every i, any two loyal generals use the same value v_i
- If the ith general is loyal, then the value that they send must be used by every loyal general as the value of v_i

Byzantine Generals Problem

- Generals communicate via messengers
- Commander general sends an order to n-1 lieutenant generals:
- **IC1:** All loyal lieutenants obey the same order
- **IC2:** If the commanding general is loyal, then every loyal lieutenant obeys the order he sends
- Can solve original problem by using Byzantine Generals solution to acquire information for each general i
  - Other generals act as lieutenants

Byzantine Generals Problem

- IC1 is agreement
- IC2 is validity

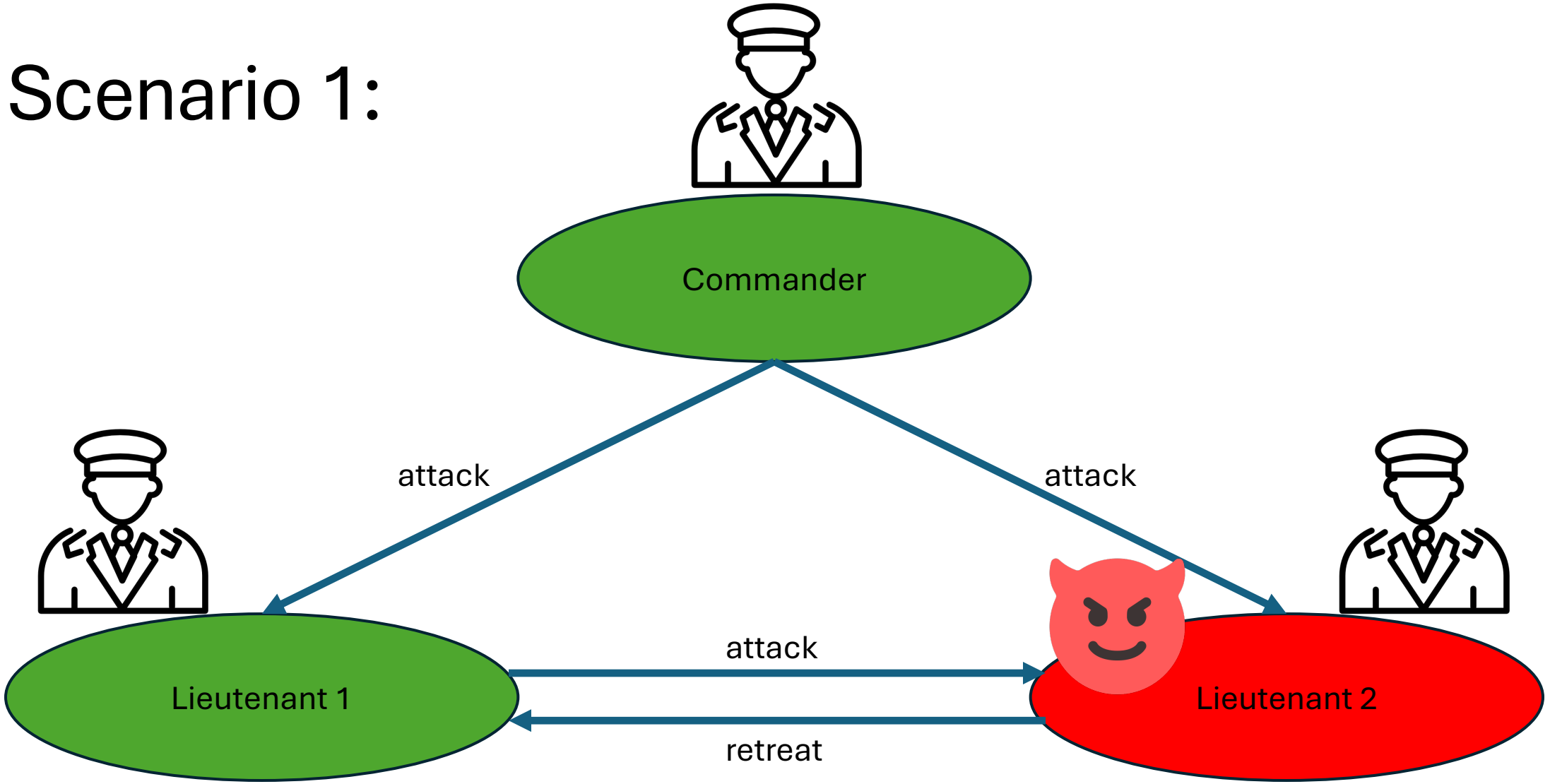
The scenario

- 3 generals, two loyal and one traitor general
- Only two possible decisions: attack and retreat
- Communication through oral messages

What are oral messages?

- A1) Every message that is sent is delivered correctly
- A2) The receiver of a message knows who sent it
  - Cannot prove to others who sent the message
  - Implies authenticated channel
- A3) Absence of a message can be detected
  - Synchronous Network (FLP)

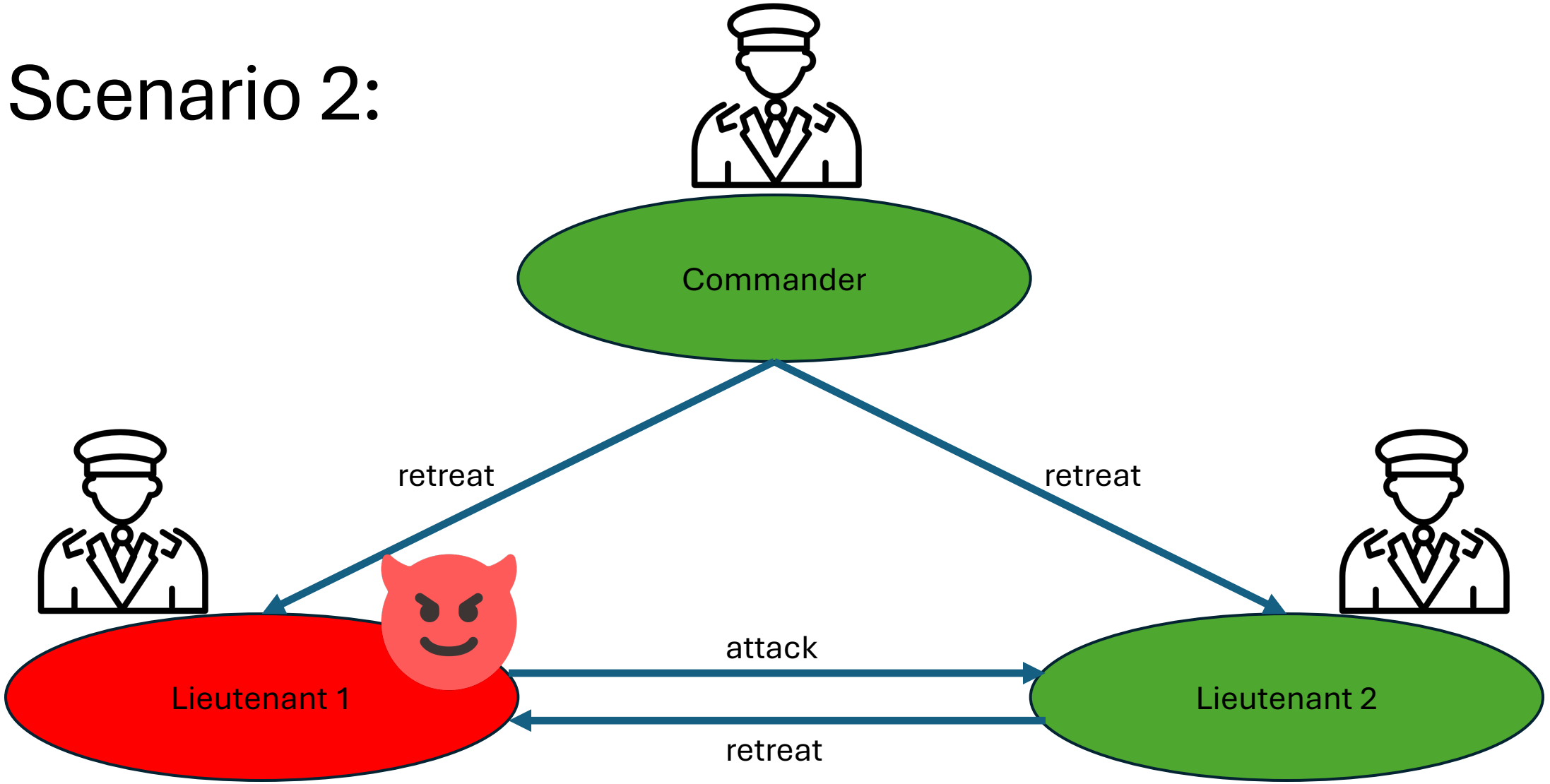
Scenario 1:



Must obey order and attack

Sees {attack, retreat}

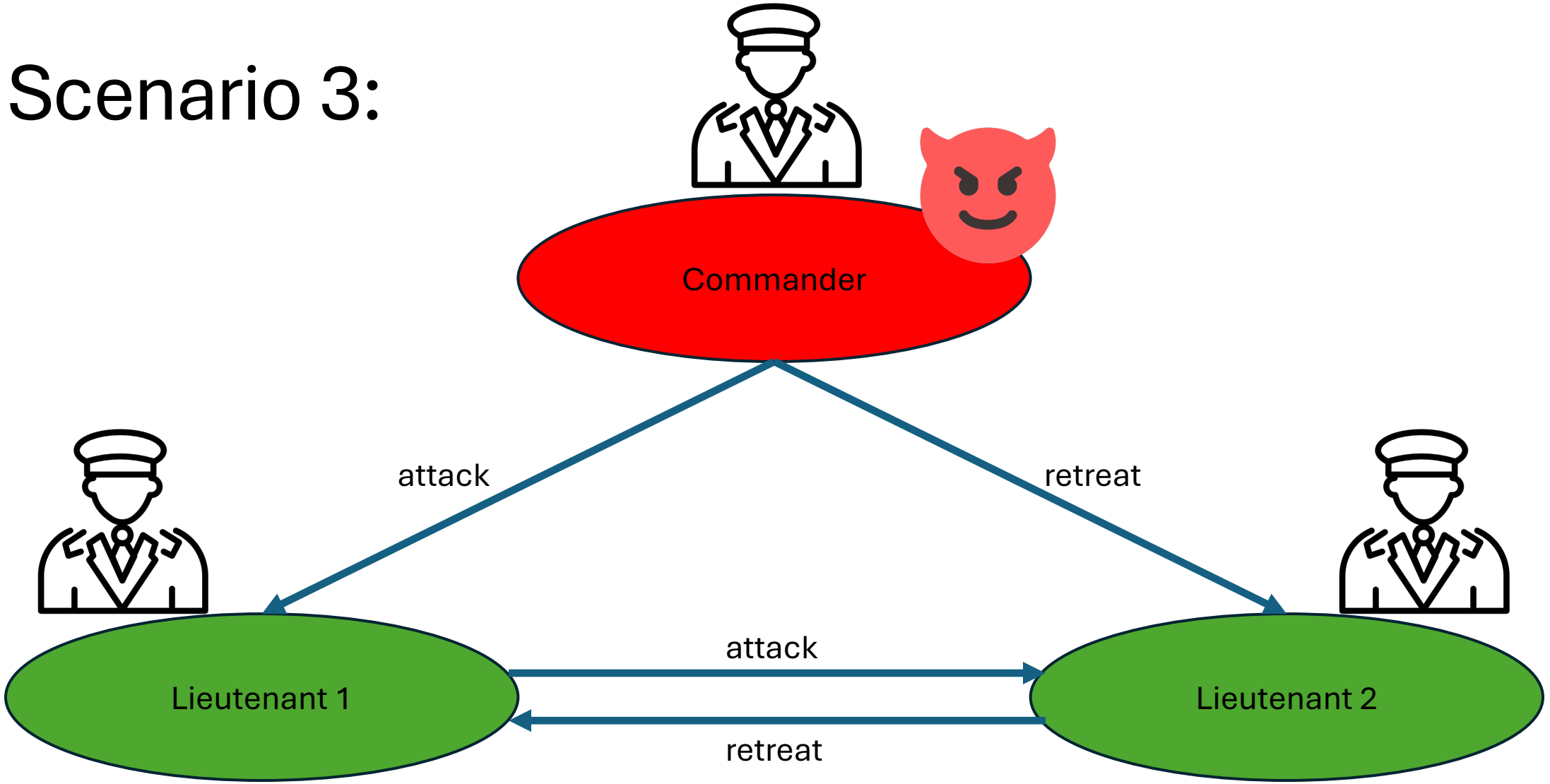
Scenario 2:



Must obey order and retreat

Sees {retreat, attack}

Scenario 3:



Must obey order and attack

Sees {attack, retreat}

Must obey order and retreat

Sees {retreat, attack}

Impossibility Result!

- Scenario 1 and Scenario 3 are indistinguishable to Lieutenant 1
- Scenario 2 and Scenario 3 are indistinguishable to Lieutenant 2
- Thus, in Scenario 3 IC1 (agreement) is violated!
  - In Scenario 3, equivocation attack took place

Impossibility Result!

Through proof by contradiction:

- In synchronous network with authenticated channels
- No solution exists with fewer than **3f + 1 generals**, where **f = number of traitors**

Solution with Oral Messages

A solution with oral messages

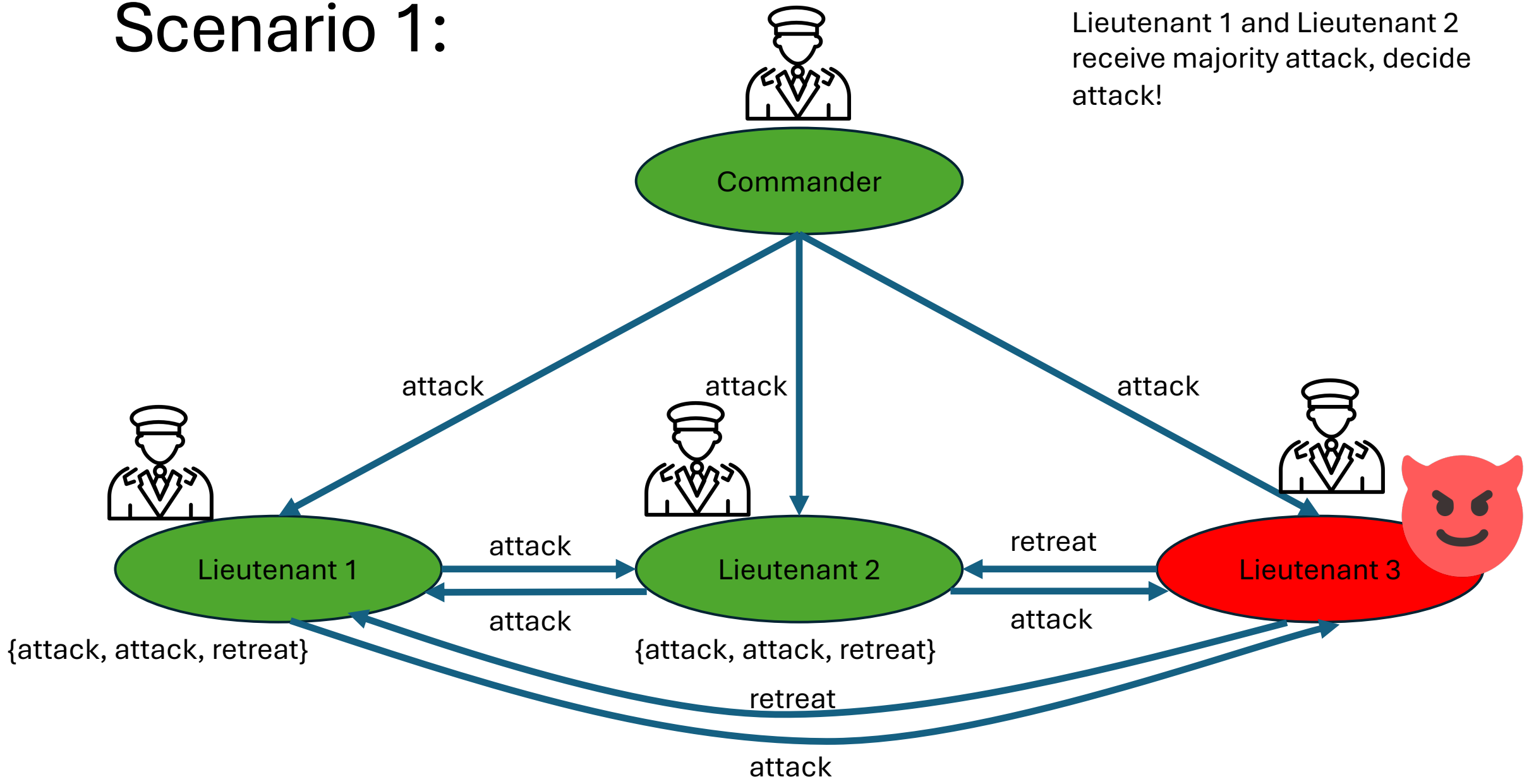
- Need $n \geq 3m+1$ generals
- Recursive algorithm:
- Algorithm OM(m) where m is number of traitors
- Base case (m=0):
  - Commander sends decision to all
  - Lieutenant uses received decision or RETREAT (default)

A solution with oral messages

- Recursive step: m > 0:
  - Commander sends decision to all
  - v_i : value that Lieutenant i receives from commander (or retreat)
  - **OM(m-1):** All lieutenants broadcast their value v_i to all other lieutenants
  - v_j: value that Lieutenant i receives from lieutenant j (or retreat)
  - Lieutenant i uses value majority (v_1, … v_j, … v_{n-1})
- Continuously broadcast values until base case: OM(m = 0)

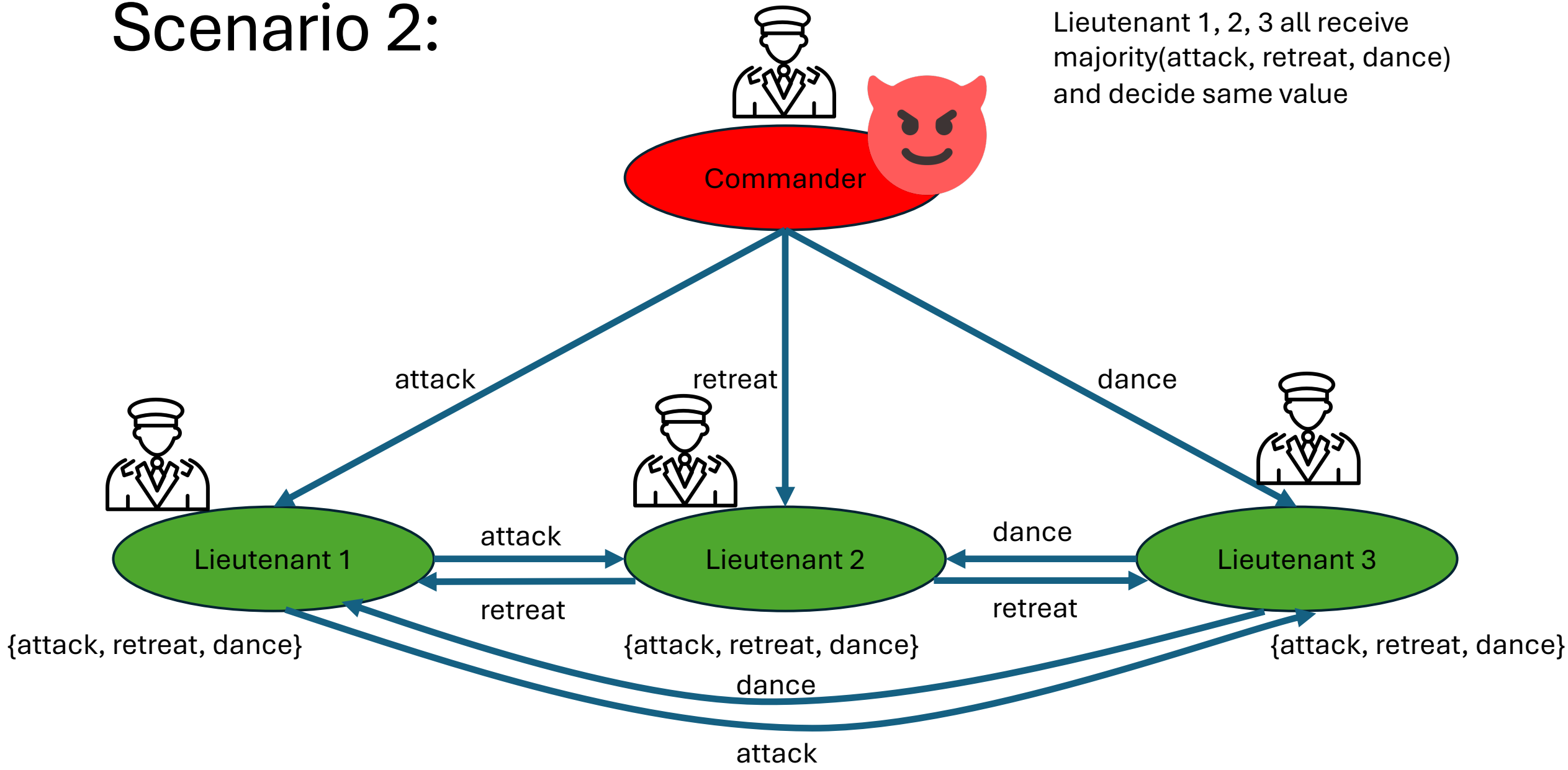
Scenario 1:

Lieutenant 1 and Lieutenant 2 receive majority attack, decide attack!



Scenario 2:

Lieutenant 1, 2, 3 all receive majority(attack, retreat, dance) and decide same value



Solution with Signed Messages

Signed Messages

- Can sign messages and send signed message to other generals
- Signatures are unforgeable for loyal generals
  - Any alteration of the signature can be detected
- Anyone can verify authenticity of general's signature
- No assumptions about traitor general's signature!
  - Other traitor generals can forge traitor general signature
- Can prove to others who sent the message

A solution with signed messages

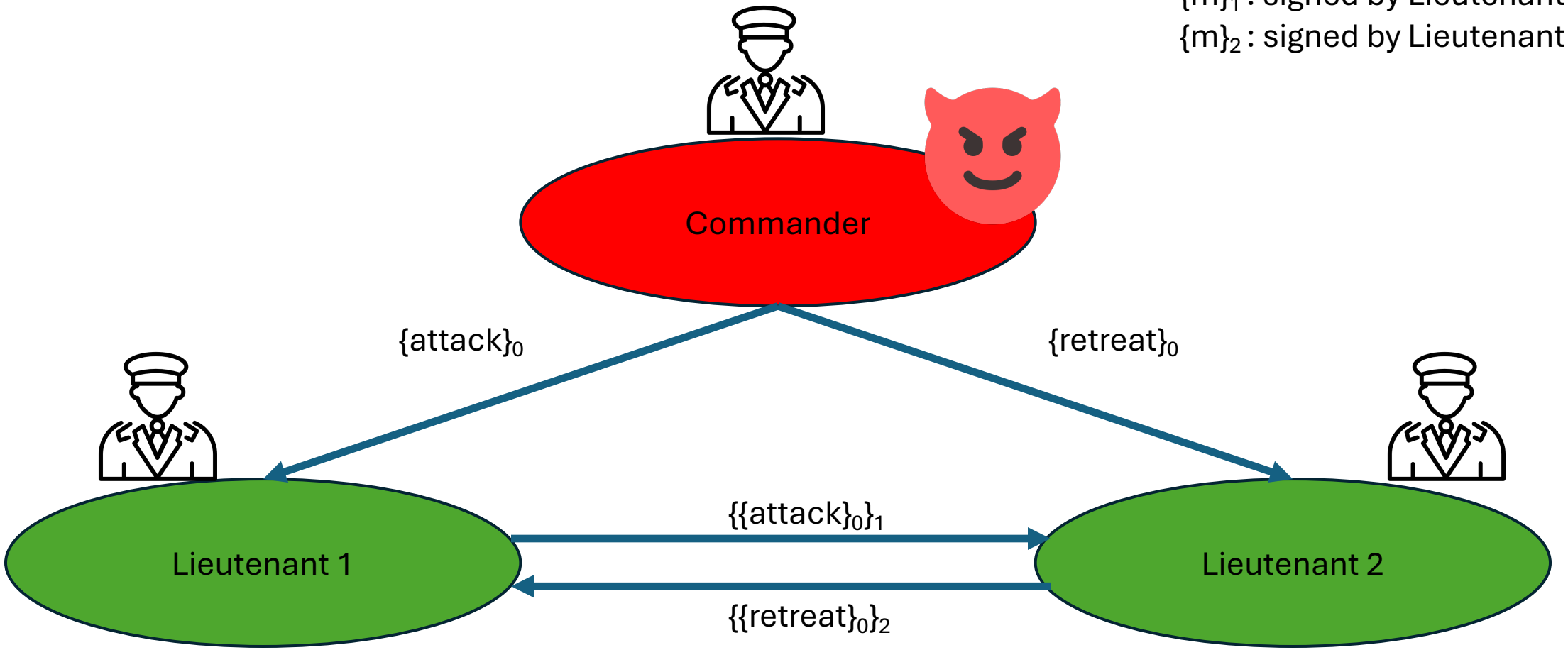
- Commander signs and sends value to all lieutenants
- Initially $V_i = \emptyset$
- $\mathrm{SM(m)}$: For each lieutenant:
  - First order directly from commander:
    - $V_i = \{v\}$
    - Appends their own signature, broadcasts to all other lieutenants
  - Receives a message v:0:j_1...j_k s.t. $v \notin V_i$
    - $V_i \cup v$
    - If k < m: sends to all other lieutenants except j_1...j_k

A solution with signed messages

- When lieutenant i will receive no more messages, obeys *choice*(V_i)
  - *choice*(V_i)
    - $V_i = \emptyset$: Retreat
    - $|V_i| = 1$: Whatever value is in V_i

Signed Messages Example

$\{m\}_0$: signed by Commander
$\{m\}_1$: signed by Lieutenant 1
$\{m\}_2$: signed by Lieutenant 2



$[\{attack\}_0, \{\{retreat\}_0\}_2]$

Choice(retreat, attack)

Discover commander is a traitor!
Detect Equivocation!

$[\{retreat\}_0, \{\{attack\}_0\}_1]$

Choice(retreat, attack)

Discussion

- What is required for algorithms OM(m) and SM(m) to work when generals cannot send to all other generals?
  - Answer: the graphs showing which generals can communicate with other generals must be m-regular
- What are some difficulties adapting the Byzantine Generals problem to real computing systems?
  - Message passing system must satisfy oral/signed message requirements (A1 to A3/A4)
  - Acquiring a meaningful input from input devices

Byzantine Fault Tolerance Requirements

(f = traitors/faulty, n=total)	Synchronous	Partial-Synchrony	Asynchronous
Oral Messages: Impossibility	$n \leq 3f$ (LSP80)		$f \geq 1$ (FLP)
Oral Messages: Possible	$n \geq 3f + 1$ (LSP80)		
Signatures: Impossibility			$f \geq 1$ (FLP)
Signatures: Possible	$n \geq f+1$ (LSP80)	$n \geq 3f+1$ (CL99)	

*slide inspiration taken from https://www.cs.cornell.edu/courses/cs6410/2010fa/lectures/24-byzantine.pdf

Practical Byzantine Fault Tolerance Intro

- Miguel Castro (1)
  - MIT PhD in CS
  - Laboratory for CS, MIT
  - Microsoft Research
  - OpenAI
- Barbara Liskov (2)
  - Stanford PhD in CS
  - Laboratory for CS, MIT
  - Head of the Programming Methodology Group
  - Turing Award 2008

1



2



PBFT Setup:

- Form of state machine replication that tolerates byzantine faults
  - Primary/Backup architecture
  - Sequence number for requests dictates order of execution
- Asynchronous system
- Signs messages
  - Assumes signatures/MACs are unbreakable
- Requires ≥ 3f+1 replicas for safety and liveness
  - Safety: Replicated service satisfies linearizability
  - Liveness: service is live if network is synchronous
- Replicas must be deterministic and must start in the same state

Why is PBFT Practical?

- Most similar previous work is Rampart and SecureRing
  - PBFT does not require synchrony for correctness
- Improves performance
  - One message round trip for read only transactions
  - Two for read-write operations
- Public key cryptography only used when faults occur
  - Main bottleneck in Rampart

Normal Operation

- Client sends request to primary
- Primary sends pre-prepare request to all replicas
- If replica accepts pre-prepare, broadcast prepare
- Replica considers request prepared if:
  - Pre-prepare & 2f prepared msgs received
- If request prepared, broadcast commit
  - Request is committed iff request is prepared, and received 2f+1 commit messages
- Once request is committed, can execute request and reply to client

BFT Work After PBFT

- Query/Update (SOSP’05)
- Aardvark (NSDI’09)
- Zyzzyva (2010)
- Tendermint (2018)
- HotStuff (PODC’19)

Discussion

- What is the mechanism to allow election of a new primary in case the old one fails?
  - View Changes
- What service did the authors of PBFT implement through their protocol?
  - A network file system

Works cited

- Shostak: https://commons.wikimedia.org/wiki/File:Robert-shostak-2019-thumbnail.jpg
- Lamport: https://en.wikipedia.org/wiki/File:Leslie_Lamport.jpg
- Devil icon: https://www.flaticon.com/free-icons/devil" title="devil icons
- Castle icon: https://www.flaticon.com/free-icon/castle_6161670
- General icon: https://www.flaticon.com/free-icon/general_1461815
- Miguel Castro: https://www.microsoft.com/en-us/research/wp-content/uploads/2017/01/avatar_user__1483926285.jpg
- Barbara Liskov: https://commons.wikimedia.org/wiki/File:Barbara_Liskov_MIT_computer_scientist_2010.jpg